ANALYSIS OF PERFORMANCE CHARACTERISTICS OF THE PROCESSOR FOR VARIED CACHE SIZES USING SIMPLESCALAR AND SPEC 2000 BENCHMARK SUIT

NITIN THOTA-2354187

Abstract:

The idea of this project is to examine the performance evaluation of SPEC2000 benchmarks using the Simple Scalar simulator. This helps us understand how different parameters affect performance by using benchmarks from the SPEC2000 suite and evaluating them with Simple Scalar, an open-source architectural simulation. We prior on key parameters like L1 and L2 cache sizes, associativity, and block sizes, analyzing their impact on performance. The results, given through bar graphs, give us a clear picture of how these configurations influence execution metrics and provide useful insights for optimizing cache designs to improved efficiency.

Keywords: -

Simple Scalar Simulator, SPEC 2000 benchmark

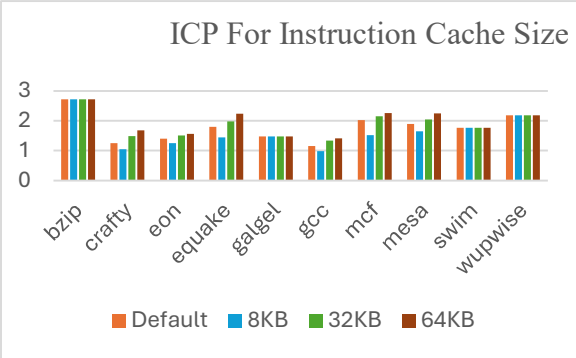
Introduction: -

In this Project we are using Simple Scalar in this simulator we are going use Spec2000 which is test Bench Suite which contains various test benchmark Such as bzip, crafty, eon, equake, galgel, gcc, mcf, mesa, swim, and wupwise, by taking this we are going to perform some sets of parameters such as cache size, associativity, and block size and but get those and viewing the analysis we will able to understand there importance and significant importance for the performances matrix here the results are presented in the in the form of bar graphs

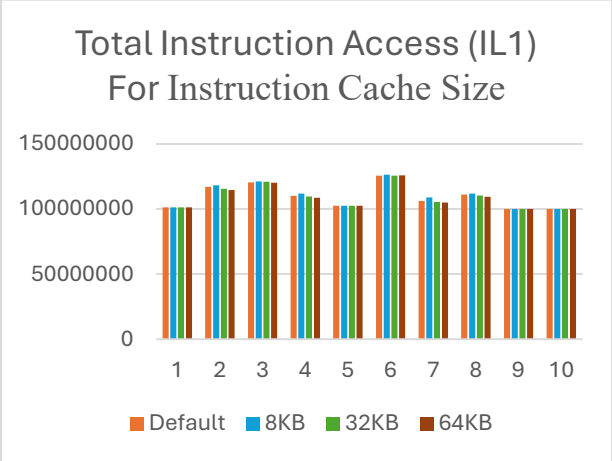
Application

Instruction Cache:

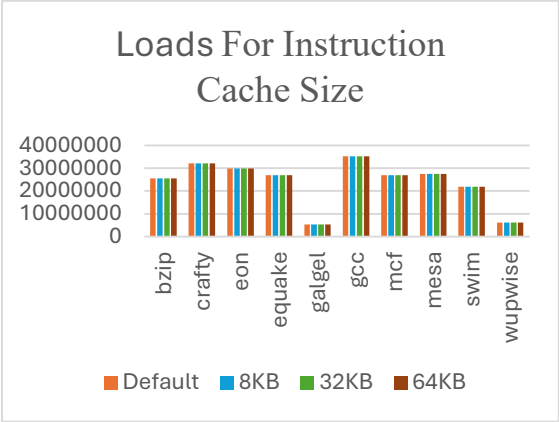
Here in this process we have various cache sizes, including 8KB,32 KB,64KB
are being tested for benchmarks performances with how theses sizes affect the main cache performance has been tested and indicated below.



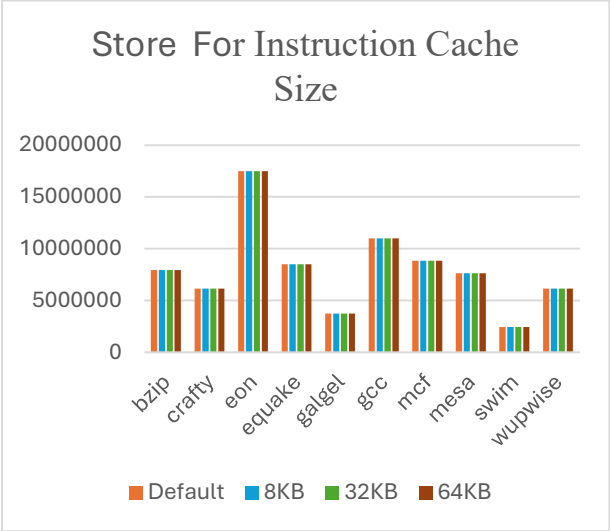
IPC for Instruction Cache Size



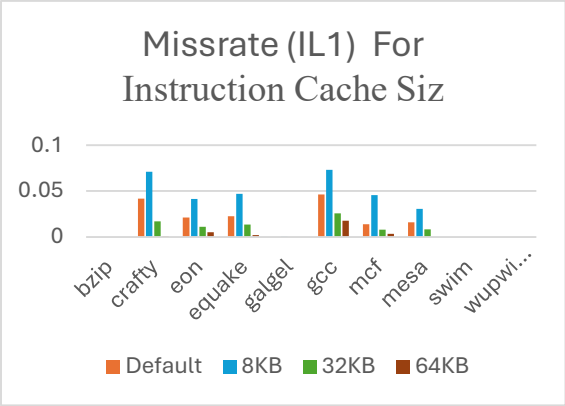
Total for Instruction Cache Size



Loads for Instruction Cache Size



Store for L1 Cache Size



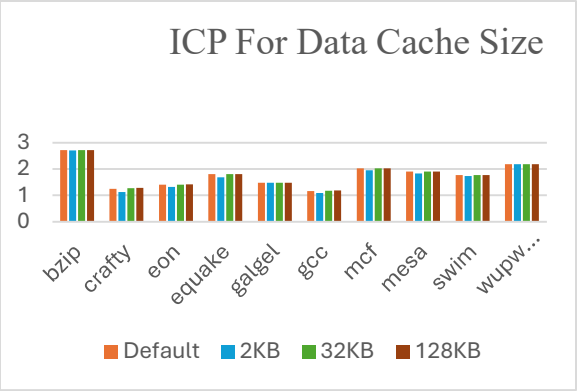
Miss Rate (IL1) for Instruction Cache Size

The results indicate that Instructions Per Cycle (IPC) improved considerably with increased cache sizes. Benchmarks such as crafty and equake show significant IPC enhancements as the cache size grows from 8KB to 64KB, demonstrating the benefits of larger caches. Regarding Instruction Access (IL1), the total accesses remain largely consistent across various cache sizes, suggesting that the program's execution path is not impacted by these changes. However, the miss rate decreases significantly with larger caches. For instance, crafty and eon experience substantial reductions in miss rates when moving from 8KB to 64KB. These results highlight the importance of larger cache sizes, like 64KB, in boosting performance metrics such as IPC and reducing miss rates, thereby improving execution efficiency for SPEC2000 benchmarks.

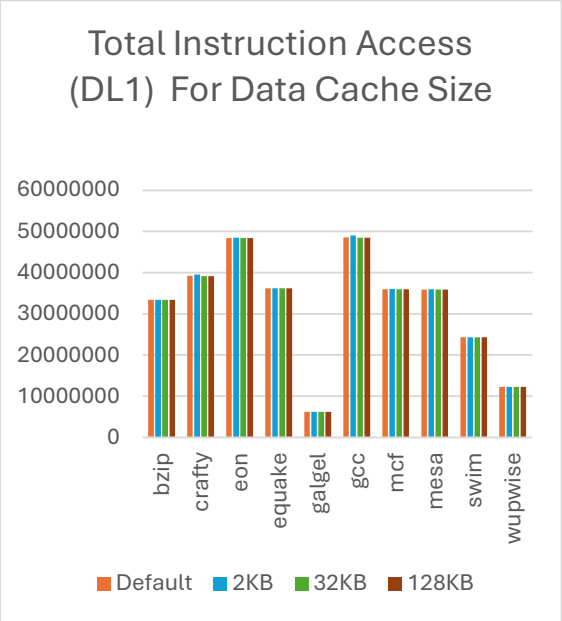
<u>Data Cache: -</u>

Various cache sizes, including 2KB, 32KB,

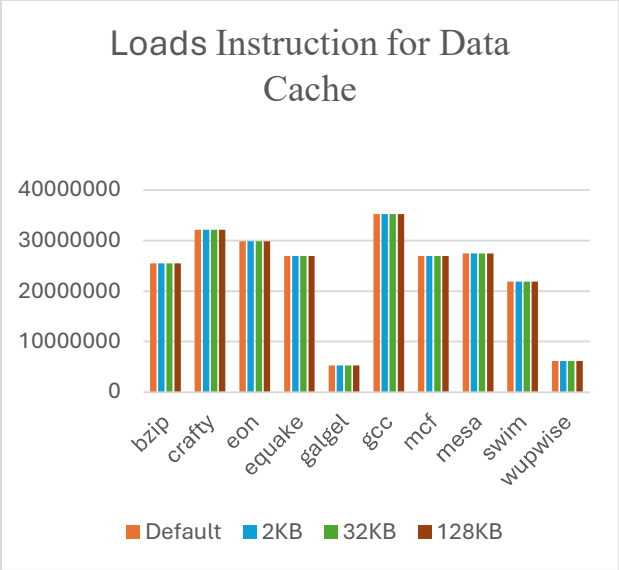
and 64KB, were tested to evaluate benchmark performance and analyze how cache size impacts key cache performance parameters.



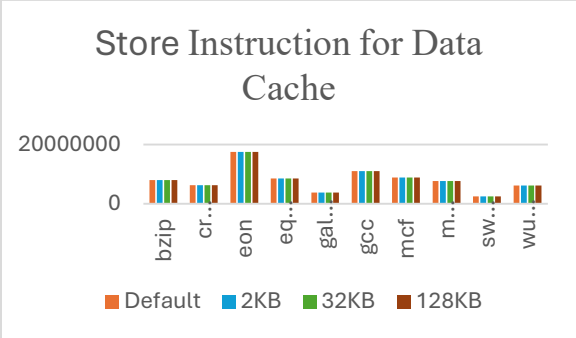
ICP for Data Cache



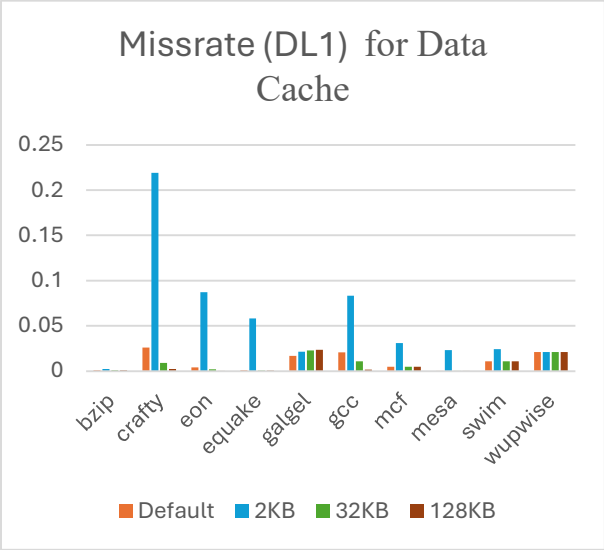
Total for Data Cache



Load instruction for Data Cache



Store Instruction for Data Cache



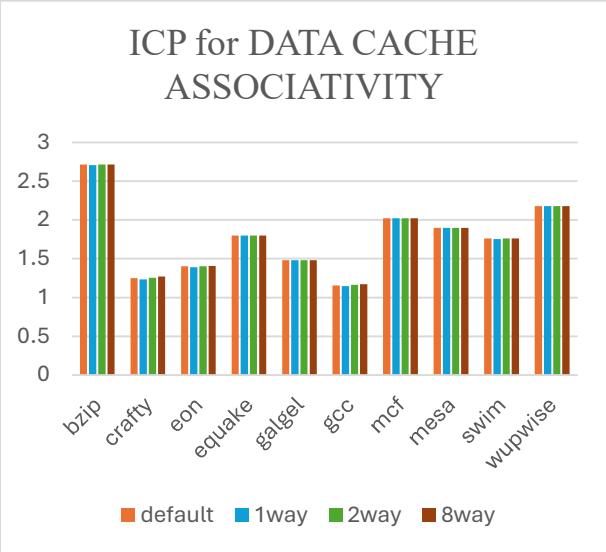
Missrate (DL1) for Data Cache

From the results, it is evident that smaller cache sizes, such as 2KB, tend to have higher miss rates. However, as cache size increases, particularly between 32KB and 128KB, the miss rate decreases significantly. In some cases, benchmarks like **bzip** and **swim** exhibit minimal performance changes across varying cache sizes, likely due to their workload characteristics that are less dependent on cache optimization. Additionally, while the total data accesses remain relatively consistent regardless of cache size, larger caches enhance the efficiency of data retrieval, improving overall performance.

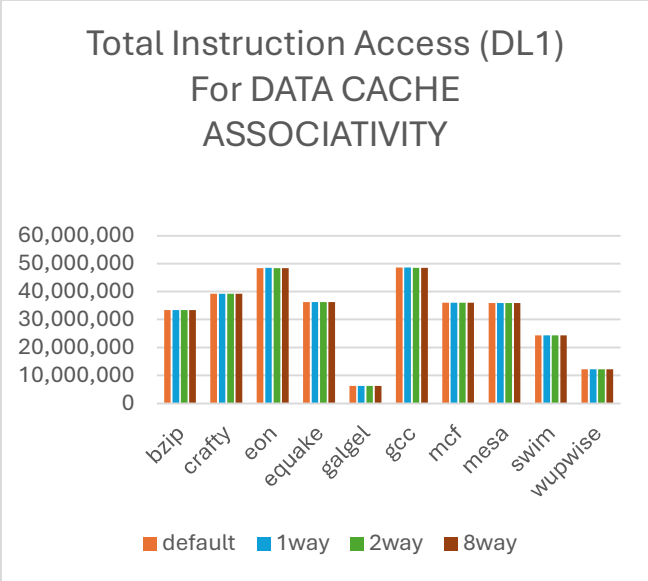
<u>Data Cache Associativity: -</u>

This analysis includes various cache associativity configurations, such as 1-way, 2-way, and 8-way associative caches. The performance of different benchmarks has been evaluated using these configurations alongside several key performance metrics.

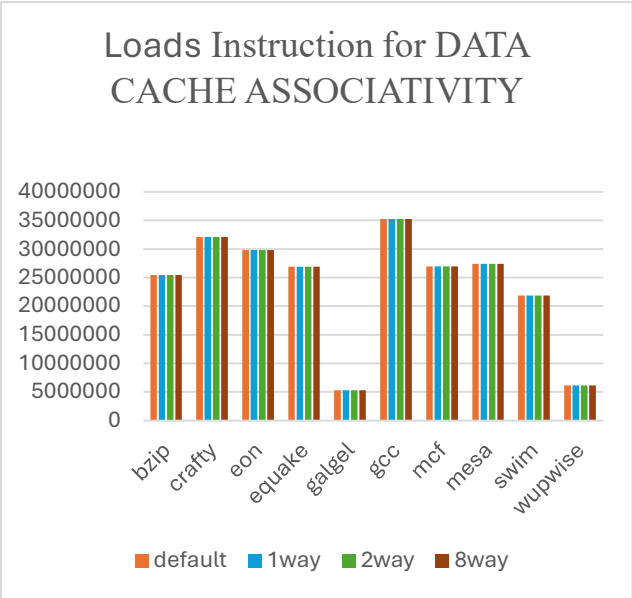
.



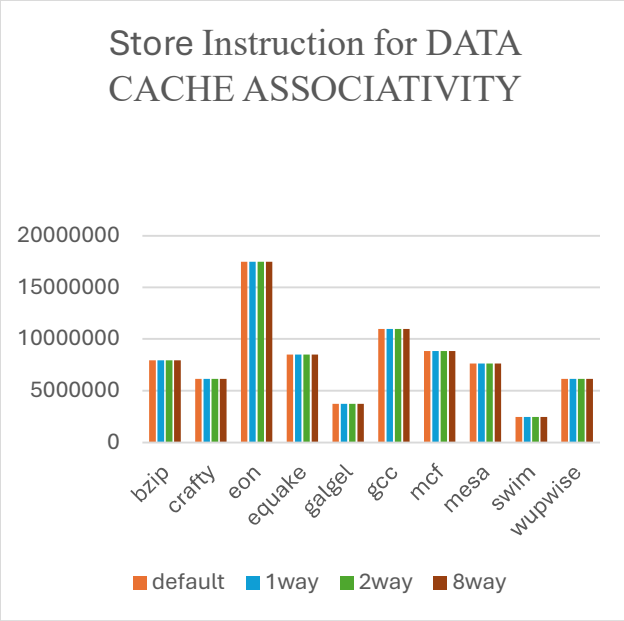
ICP For Data Cache Associativity



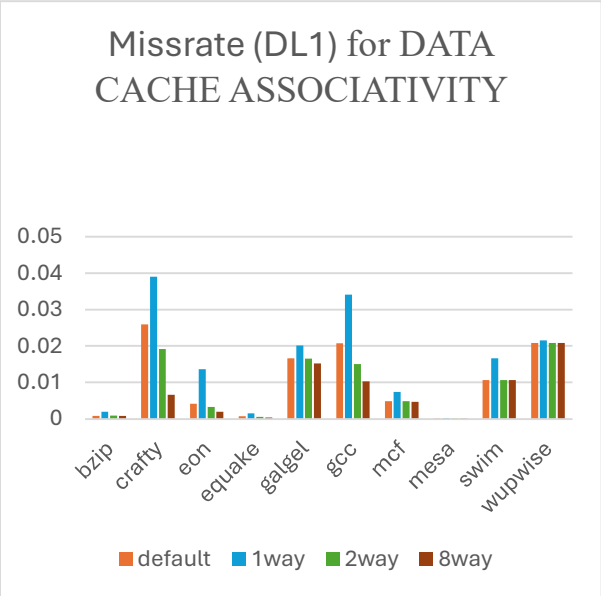
Total Instruction access



Loads for data Cache Associativity



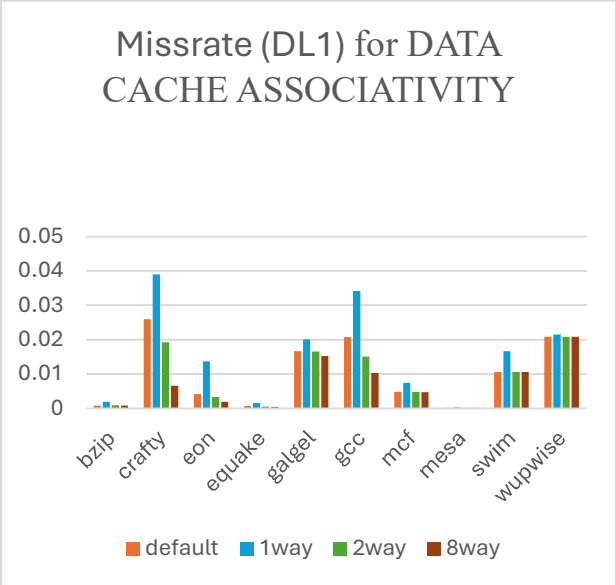
Store For Data Cache Associativity



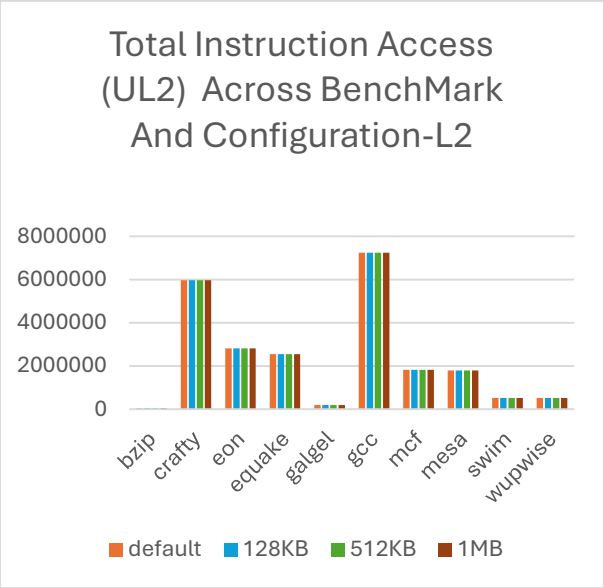
Missrate For Data Cache Associativity

<u>L2 CACHE SIZE</u>

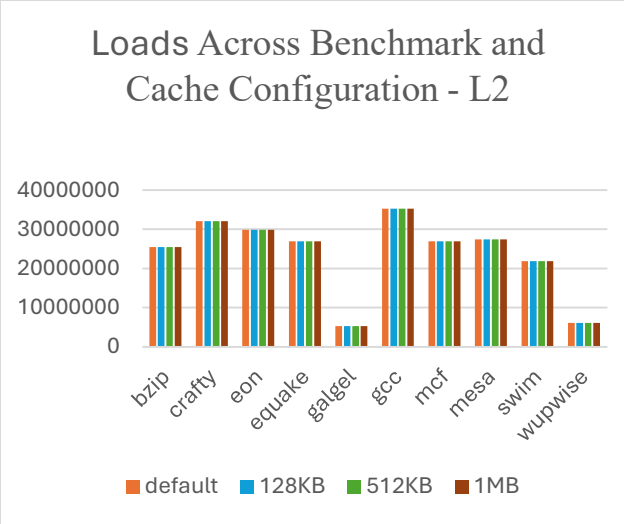
The results clearly show that data cache associativity, ranging from 2-way to 8-way, leads to a gradual increase in IPC for benchmarks like gcc and eon. Higher associativity reduces conflict misses, contributing to improved performance. There is also a clear inverse relationship between miss rate and associativity; as associativity increases, the miss rate decreases. However, certain benchmarks, such as bzip and swim, exhibit minimal changes with varying associativity, suggesting that their performance is influenced more by factors like data locality or block size rather than associativity.



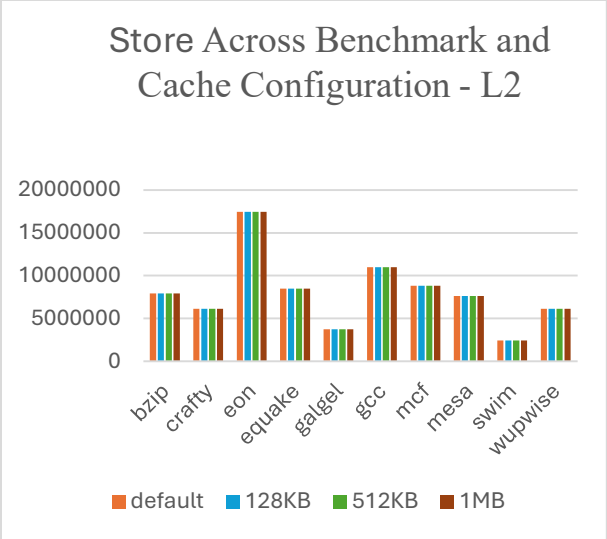
ICP for Unified L2 Cache



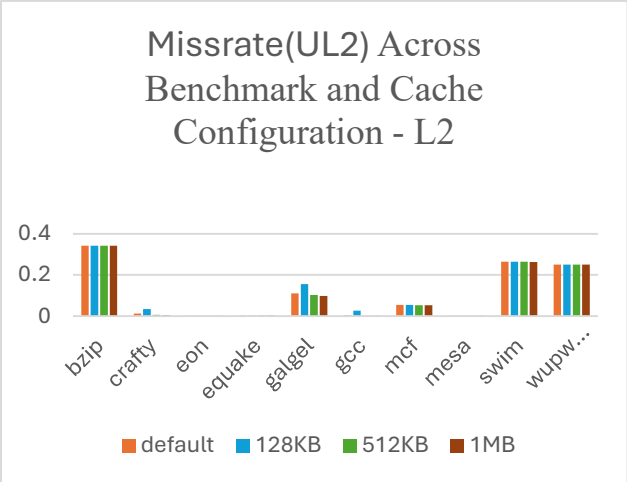
Total Instruction Access (UL2) For – Unified L2



Loads Instruction For – Unified L2



Store Instruction For Unified L2

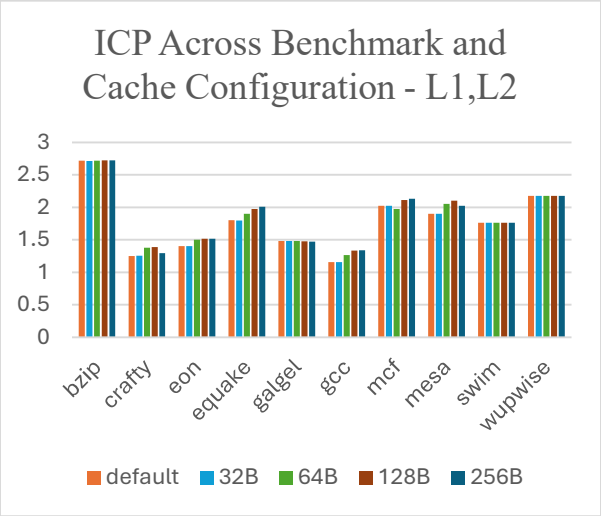


Missrate for Unified L2

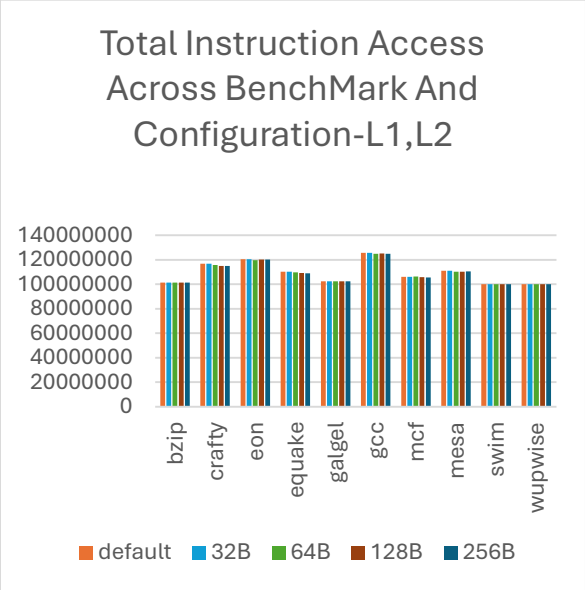
The results clearly demonstrate the impact of unified L2 cache size, ranging from 128KB to 1MB, on IPC improvement. For instance, benchmarks like **crafty** and **gcc** show noticeable performance enhancements due to reduced memory latency. However, not all benchmarks are equally affected; **bzip** and **mcf** experience minimal changes with varying cache sizes. Additionally, consistency across benchmarks highlights that workloads like **bzip** and **swim**, which

rely less on cache optimization, exhibit minimal performance variation. In contrast, benchmarks such as **crafty** and **equake** significantly benefit from larger cache sizes, achieving reduced miss rates and higher IPC.

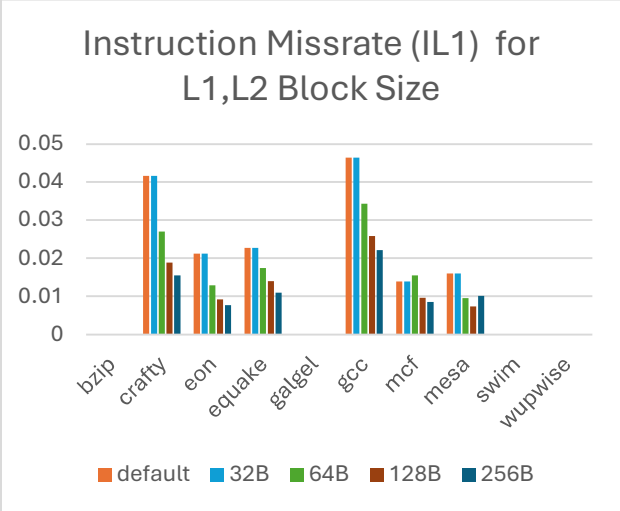
<u>L1,L2 Cache Block Size:-</u>

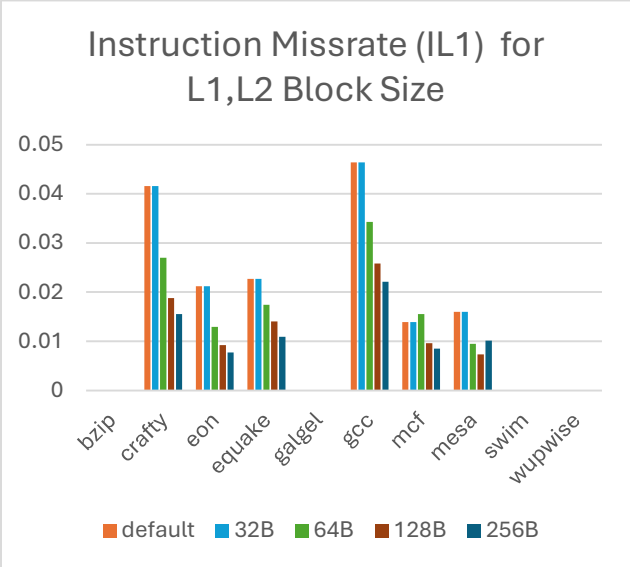


IPC For L1,L2 Block Size

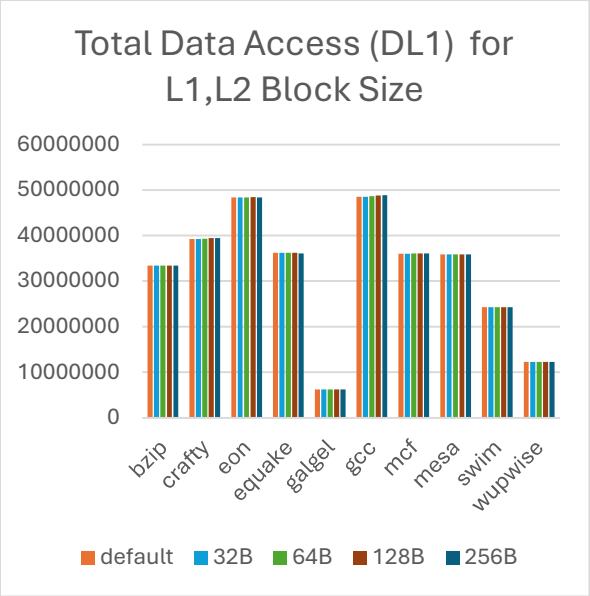


Total Instruction Access for L1,L2 Block Size

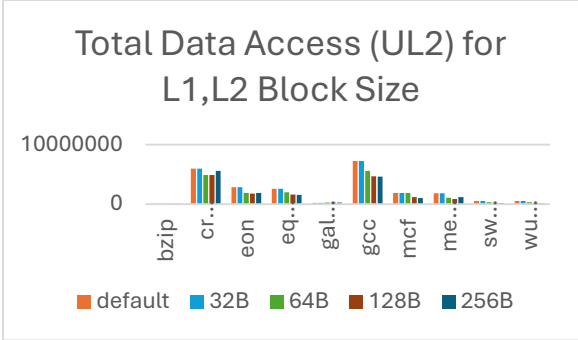




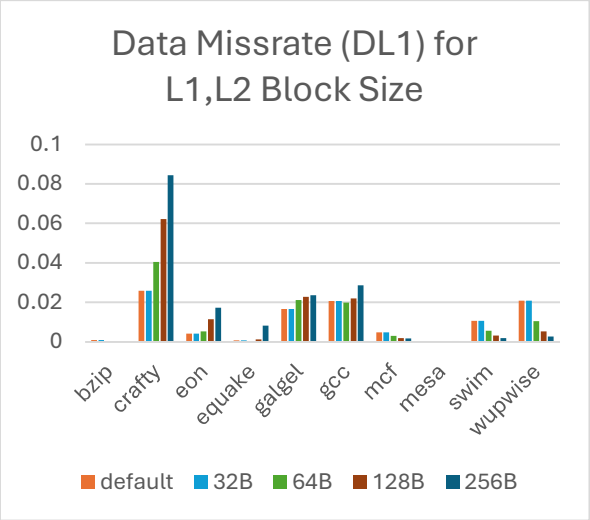
Instruction Missrate (IL1) For L1,L2 Block Size



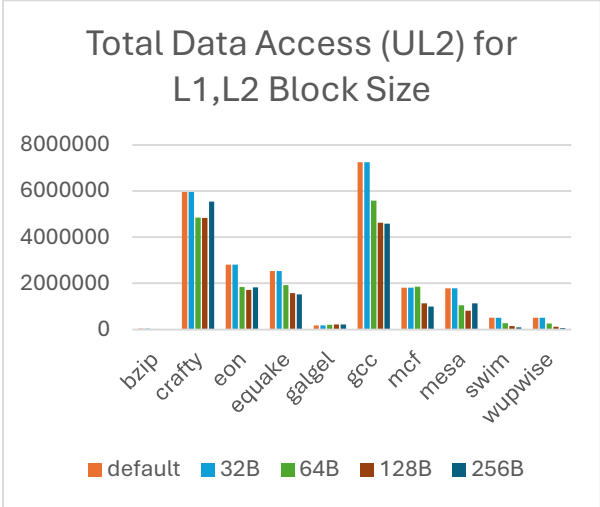
Total Data Access (DL1) For L1,L2 Block Size



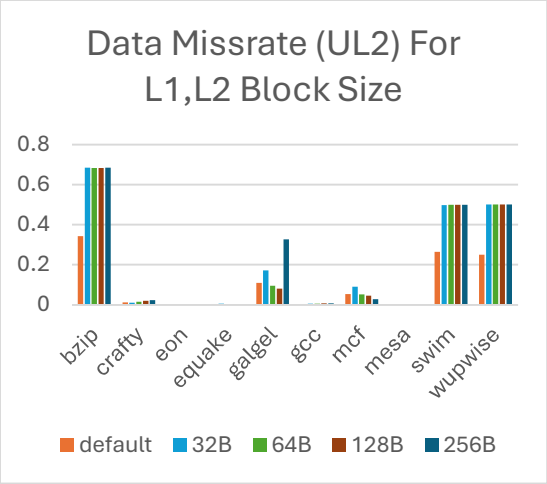
Total Data access (UL2) for L1,L2 Block Size



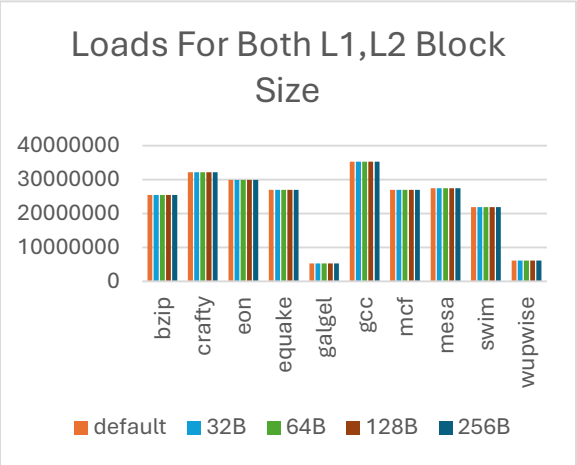
Data Missrate (DL1) For L1,L2 Block Size



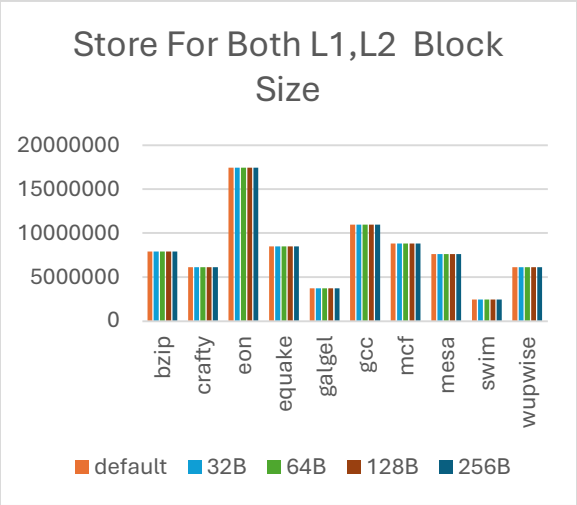
Total Data Access (UL2) For L1,L2 Block Size



Data Missrate (UL2) For L1,L2 Block Size



Loads For Both L1,L2 Block Size



Store Both L1,L2 Block Size

The results clearly indicate that increasing the L1 block size and associativity leads to significant improvements in IPC for benchmarks like **equake** and **gcc**, highlighting enhanced performance due to better utilization of spatial locality. Changes in L2 block size also contribute positively, with benchmarks like **crafty** showing gradual IPC improvements from 64B to 128B, though the impact is less pronounced compared to L1. Regarding miss rates, L1 miss rates decrease significantly with larger block sizes, especially from 64B to 256B, as seen in benchmarks like **eon** and **galgel**, demonstrating improved data prefetching and reuse. Conversely, L2 miss rates for larger block sizes, such as 256B, show slight increases in some benchmarks (e.g., **bzip** and **equake**), suggesting diminishing benefits and possible over-fetching of unnecessary data.

<u>Conclusion: -</u>

In this course project, we successfully implemented various SPEC 2000 benchmarks in the SimpleScalar simulator. Throughout this process, we analyzed the performance of several fundamental cache configuration factors, including L1 and L2 cache sizes, associativity, and block size. Utilizing the SPEC 2000 benchmark suite, we evaluated parameters such as the number of load/store instructions, cache miss rates, and instructions per cycle (IPC). By doing so, we were able to obtain, the results and compare it effectively.